

# Layout Performance Analysis

## Mobile-First vs Desktop-First CSS

---

*Full Stack Development Internship Report*

Submitted by:

**Tayyaba Shakoor**

*Full Stack Development Intern*

**Prosensia**

05 June 2026

## 1. Mobile-First vs Desktop-First CSS

| Approach      | Media Query | Default Target           |
|---------------|-------------|--------------------------|
| Mobile-First  | min-width   | Mobile (smallest screen) |
| Desktop-First | max-width   | Desktop (largest screen) |

### Mobile-First Example (Tailwind Default)

```
/* Base: mobile */
.text { font-size: 14px; }

/* Tablet and up */
@media (min-width: 768px) { .text { font-size: 16px; } }

/* Desktop and up */
@media (min-width: 1024px) { .text { font-size: 18px; } }
```

### Desktop-First Example

```
/* Base: desktop */
.text { font-size: 18px; }

/* Tablet and down */
@media (max-width: 1023px) { .text { font-size: 16px; } }

/* Mobile and down */
@media (max-width: 767px) { .text { font-size: 14px; } }
```

**Winner: Mobile-First — less CSS on mobile devices, better performance.**

## 2. Browser Compilation Rules

The browser processes CSS from top to bottom. For min-width (mobile-first), smaller screens ignore larger breakpoints, saving bandwidth. For max-width (desktop-first), mobile devices still download desktop CSS first — wasted bytes.

| Viewport       | Mobile-First      | Desktop-First                      |
|----------------|-------------------|------------------------------------|
| Mobile (375px) | Applies base only | Applies base + all max-width rules |

| Viewport         | Mobile-First                       | Desktop-First     |
|------------------|------------------------------------|-------------------|
| Desktop (1200px) | Applies base + all min-width rules | Applies base only |

### 3. Cumulative Layout Shift (CLS)

CLS measures unexpected content shifting while the page loads.

| CLS Score  | Rating            |
|------------|-------------------|
| 0 – 0.1    | Good              |
| 0.1 – 0.25 | Needs Improvement |
| > 0.25     | Poor              |

#### Common Causes

---

- Images without width/height attributes
- Ads or embeds without reserved space
- Web fonts loading late
- Dynamic content injected above existing content

#### Prevention

---

```
// Good


<div className="min-h-[200px]">
  {/* Reserved space for dynamic content */}
</div>
```

### 4. Google Lighthouse Performance Metrics

Core Web Vitals are Google ranking factors that measure real-world user experience.

| Metric | Measures                                 | Good Score    |
|--------|------------------------------------------|---------------|
| LCP    | Loading speed (Largest Contentful Paint) | < 2.5 seconds |
| FID    | Interactivity (First Input Delay)        | < 100 ms      |

| Metric | Measures                                   | Good Score |
|--------|--------------------------------------------|------------|
| CLS    | Visual stability (Cumulative Layout Shift) | < 0.1      |

## How Unoptimized Layouts Hurt Scores

---

| Problem             | Effect                            |
|---------------------|-----------------------------------|
| No image dimensions | CLS increases                     |
| Desktop-first CSS   | Extra download overhead on mobile |
| No breakpoints      | Broken mobile layout              |
| Layout reflows      | Slow rendering and poor LCP       |

## 5. Unoptimized Layout Reflows

### What is reflow?

A reflow is the browser recalculating element positions and sizes. It is one of the most expensive operations in the rendering pipeline.

### Bad Code — Multiple Reflows

---

```
element.style.width = '100px'; // reflow
element.style.height = '100px'; // reflow
element.style.padding = '10px'; // reflow
// Each line forces the browser to recalculate layout
```

### Good Code — Single Reflow

---

```
element.classList.add('box-styles'); // One reflow only
```

### In React

---

```
// Bad - 100 re-renders
for (let i = 0; i < 100; i++) setCount(i);

// Good - 1 re-render
setCount(100);
```

## 6. Best Practices Summary

| Practice                            | Why                                        |
|-------------------------------------|--------------------------------------------|
| Use mobile-first (Tailwind default) | Better mobile performance, less wasted CSS |
| Set image dimensions                | Prevent CLS score from increasing          |
| Use relative units (rem, %, vw)     | Scales correctly across all devices        |
| Reserve space for dynamic content   | No unexpected layout shift on load         |
| Batch DOM reads/writes              | Reduce number of reflows                   |
| Use CSS transforms for animations   | GPU accelerated — causes no reflow         |

## 7. Conclusion

| Point                         | Summary                                    |
|-------------------------------|--------------------------------------------|
| Mobile-first vs Desktop-first | Mobile-first wins (Tailwind default)       |
| CLS                           | Keep below 0.1 by setting image dimensions |
| Lighthouse                    | LCP (<2.5s), FID (<100ms), CLS (<0.1)      |
| Reflows                       | Batch operations, avoid per-property loops |

### One-Line Summary

*"Use mobile-first Tailwind breakpoints, set image dimensions to prevent CLS, and batch DOM updates to avoid layout reflows."*

---

Report Prepared By: Tayyaba Shakoor  
Date: 05 June 2026 | Internship: Prosensia — Full Stack Development